Module 19 – Troubleshooting

ProTech
protechtraining.com



Observe before you alter.



⚠ A restart can restore service and simultaneously destroy the explanation.

2 Route Symptoms to the Right Evidence

Map symptom categories to a bounded log, trace, and dump collection strategy—not copying every profile file.

1. Symptom Category	2. What to Collect	3. Bounded Strategy	4. Output (Not Everything)
Performance degradation	Performance logs, APM traces	Time-bound window (e.g., ±15 min around event)	Compressed perf package
Authentication / authorization	Auth & access logs, audit trails	Filter to relevant users / events	Security evidence pack
Functional failure	Application logs, error details	Focus on failed transactions & correlations	Error & context package
System instability	System logs, crash dumps	Capture at failure onset (not 24/7)	Crash & system dump package

Memorable Statement:

Collect **what the hypothesis demands**, not **everything available**.

Read Basic and Native Logs with Intent

3

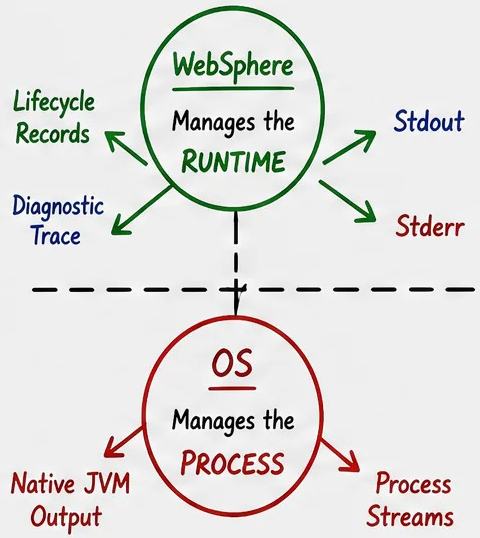
Differentiate runtime standard output, standard error, lifecycle records, diagnostic trace, and native JVM output; correct the legacy native-log oversimplification.

Memorable Statement

WebSphere manages the runtime; the OS manages the process.

		What it is	What's in it	Who writes it	Where it lives
1	Runtime Standard Output (STDOUT)	Normal program messages.	App println(), info, debug, custom output	Your application	Console / stdout
2	Runtime Standard Error (STDERR)	Program errors and exception stack traces.	Exceptions, errors, warnings, stack traces	Your application	Console / stderr
3	Lifecycle Records (SERVICE / AUDIT)	Server lifecycle, audit, and operational events.	Server start/stop, deployments, config changes, user activity	WebSphere	logs/ SystemOut.log
4	Diagnostic Trace (TRCEDATA)	Detailed diagnostic tracing for troubleshooting and performance.	Traces, entry/exit, timing, resources, performance data	WebSphere & support components	logs/ trace/<server>/
5	Native JVM Output (SYSTEM / JVM)	JVM and native libraries write to OS process streams/files.	GC logs, JIT, JNI, native crashes, core messages	JVM & native components	OS process stdout/stderr or files

The Clear Separation



Legacy Oversimplification:
"Native logs are just SystemOut.log."

→

Reality: Native output is written by the JVM/OS outside of WebSphere's lifecycle logging.

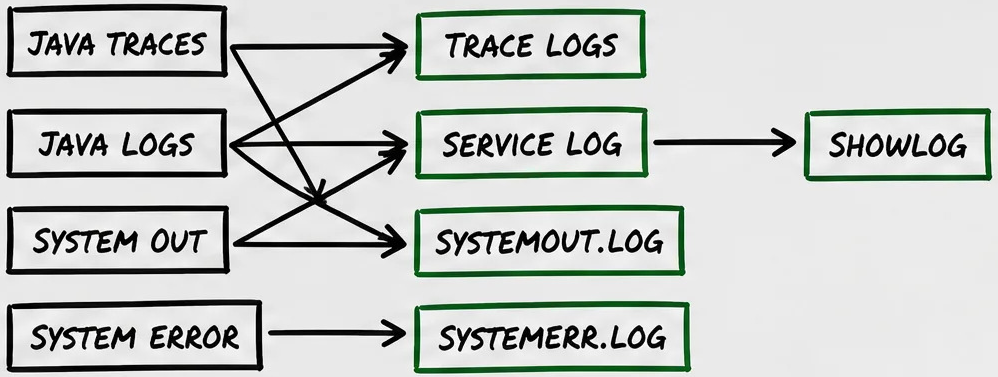
Read with intent. Know the source. Trust the signal.

Choose Basic Mode or HPEL Deliberately

4

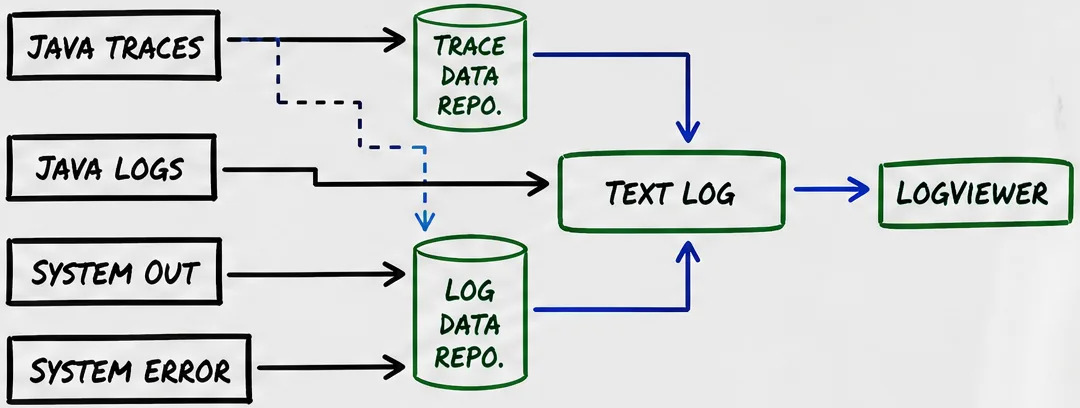
Compare the default Basic-mode plain-text path with HPEL's binary repositories, retrieval tooling, and restart requirement.

Basic Mode (Default)



- Outputs to plain-text log files
- Human-readable but hard to correlate at scale
- Manual search, limited filtering, slower triage
- No restart requirement to use

HPEL Mode



- Writes to binary repositories (trace & log data repos)
- LogViewer for fast search, correlation, and filtering
- Designed for machine performance & large-scale analysis
- RESTART REQUIRED to enable HPEL mode

Memorable Statement:

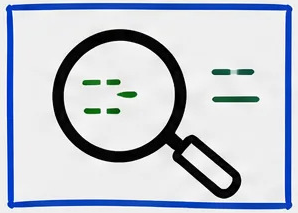
HPEL Trades Human Readability for Machine Performance and Filterable Retrieval.

Trace Narrowly, Time-Box It, Prove It



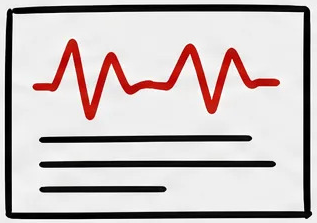
Turn diagnostic trace from a broad emergency reaction into a controlled runtime experiment with scope, output, expiry, and acceptance evidence.

1 SCOPE



- Trace narrowly.
- Target the problem.
- Minimize overhead.

2 OUTPUT



- Useful data only.
- Structured & readable.
- Actionable signals.

3 EXPIRY



TIME-BOX IT!

- Set a firm end time.
- Auto-expire & stop.
- Protect performance.

4 ACCEPTANCE



- Define success.
- Collect evidence.
- Close the loop.

5 PROVE IT



- Show before/after.
- Measure impact.
- Share the proof.



A TRACE LEFT RUNNING IS A PERFORMANCE BUG WAITING TO HAPPEN.

TREAT HANGS AND LEAKS AS DIFFERENT PROBLEMS

6

Separate potential hung-thread investigation from classloader memory-leak policy, and surface the dump, capacity, and default-setting risks.

1 POTENTIAL HUNG-THREAD INVESTIGATION



RISKS TO MANAGE

- DUMP OVERHEAD
- CAPACITY CONSTRAINTS
- DEFAULT SETTINGS

2 CLASSLOADER MEMORY-LEAK POLICY



RISKS TO MANAGE

- DUMP OVERHEAD
- CAPACITY CONSTRAINTS
- DEFAULT SETTINGS

MEMORABLE STATEMENT:

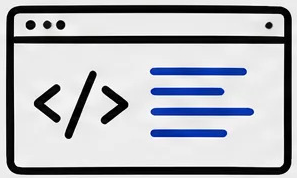
A **HANG** STOPS WORK; **A LEAK** CONSUMES SPACE.
USE DIFFERENT TOOLS FOR DIFFERENT CONSTRAINTS.

Prepare a Safe Diagnostic Change

7

Convert the legacy lab aim into a preflight plan for JVM logging, HPEL, trace, and dump tasks with control points.

1 JVM LOGGING → 2 HPEL → 3 TRACE → 4 DUMP



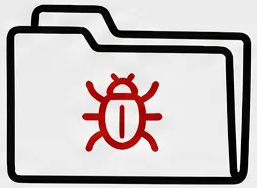
- Enable at INFO/DEBUG
- Targeted packages
- Size & rotation limits



- Collect for targeted time
- Low impact settings
- System health aware



- Targeted components
- Short duration
- Correlated IDs



- Capture when needed
- Right-sized dump
- Secure storage

CONTROL POINT 1
Low overhead
Log size bounded

CONTROL POINT 2
Resource usage OK
No alert storm

CONTROL POINT 3
Minimal overhead
Trace volume bounded

CONTROL POINT 4
Disk space OK
Access controlled



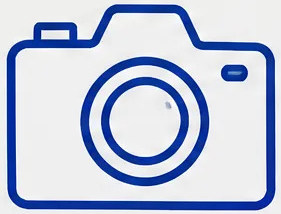
MEMORABLE STATEMENT:

DIAGNOSE THE SYSTEM WITHOUT BREAKING THE SYSTEM.

Run, Verify, and Roll Back

Execute the lab as a verifiable procedure:
capture baseline, make one controlled change,
create a known event, retrieve evidence,
then restore the intended state.

1. Capture Baseline



Document the starting state

2. Make One Controlled Change



Change with purpose

3. Create a Known Event



Trigger and observe

4. Retrieve Evidence



Collect and document

5. Restore Intended State



Roll back and validate



Rollback proves control.

CLOSE THE LOOP: PRESERVE PROOF, IMPROVE THE RUNBOOK

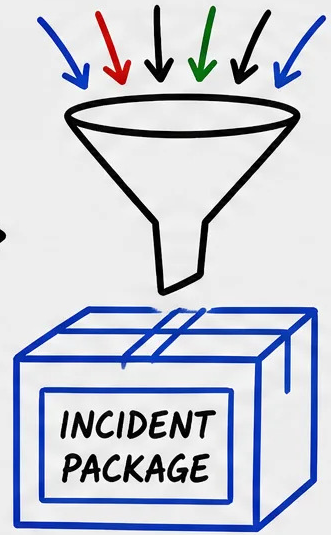
9

SYNTHESIZE LOGS, TRACES, THREAD/HEAP/SYSTEM-DUMP METADATA, VERSION BASELINE, AND RECOVERY ACTIONS INTO A REUSABLE INCIDENT PACKAGE.

1. COLLECT & PRESERVE

- LOGS
- TRACES
- THREAD/HEAP DUMPS
- SYSTEM DUMP METADATA
- VERSION BASELINE
- RECOVERY ACTIONS

2. SYNTHESIZE



- CORRELATED
- COMPLETE
- REPRODUCIBLE
- ACTIONABLE
- SHAREABLE

3. REUSE & IMPROVE



4. REUSABLE OUTPUT

INCIDENT PACKAGE

- EVIDENCE BUNDLE
- ROOT CAUSE
- RECOVERY ACTIONS
- RUNBOOK UPDATES
- LESSONS LEARNED
- SHARE & REUSE

FIX THE PROCESS, NOT JUST THE FAILURE.

BETTER RUNBOOKS, STRONGER SYSTEMS.